Pawmie Pet Care — Project Report

CS3404: GUI Programming — Mini Project (The Modern Single Page Application)

Stack: Vue 3 (Composition API) · TypeScript · Vite · Pinia · Vue Router · Tailwind CSS 4

External API: DummyJSON.com (auth, users) · randomuser.me (team page)

1. Features Implemented

Core requirements

- Strictly typed API responses — dedicated TypeScript interfaces for every fetch (DummyJSON auth, DummyJSON products, DummyJSON users, randomuser.me), no `any` in the auth or profile flows.
- Component architecture split across `views/` (route-level pages), `components/` (reusable UI), `composables/` (shared logic), and `stores/` (state) — not a single App.vue.
- Fully responsive layout (mobile / tablet / desktop) using Tailwind Grid and Flex utilities throughout.
- Data fetched live from DummyJSON: `/auth/login` for authentication, `/products` for the marketplace catalogue, `/users` for review avatars.
- Search and category filtering on the Marketplace page, plus a separate live search/filter on My Bookings (by pet, owner, or service).
- Detail view: clicking a marketplace item opens a dedicated `/marketplace/:id` route (dynamic routing).

Bonus features

- **Auth Simulation** Login via the DummyJSON `/auth/login` endpoint; access token and user profile persisted to `localStorage`; navbar and route guards react to a live "Log In / Log Out" state.
- **Cart / Persistence** Marketplace cart and orders persist across reloads via `localStorage`-backed composable state.
- **Dynamic Routing** Vue Router with parameterized routes (`/marketplace/:id`) and a wildcard 404 route.
- **Dark Mode** Full `dark:` variant coverage across the app with a persisted toggle in the navbar.

Additional polish (beyond the rubric minimum)

- **Pinia state management** — bookings are managed by a real `defineStore` (state / getters / actions), replacing an earlier module-singleton composable.
- **Toast notification system** — a global `useToast` composable + `ToastContainer.vue` gives consistent success/error feedback for bookings, cancellations, and the contact form (replacing a raw `alert()`).
- **Profile page** (`/profile`) — shows the authenticated user's account details and live booking statistics.
- **Custom 404 page** — any unmatched route now renders a branded "page not found" screen instead of a blank page.
- **Reusable composables** — form logic for the contact page (`useContactForm`) and scroll-reveal animations (`useScrollReveal`) are extracted out of view components.

2. Architecture — Component Hierarchy

The app is a single Vue Router instance mounted inside `App.vue`, which wraps every route with a shared `Navbar`, `ToastContainer`, and `Footer`. Page-level components live in `views/`; anything reused across pages is broken out into `components/` or `composables/`.

```
App.vue
├── Navbar.vue                 (auth state, dark-mode toggle, mobile menu)
├── ToastContainer.vue         (reads useToast() — global notification state)
├── <router-view>
│   ├── Home.vue
│   │   ├── Hero.vue
│   │   ├── FeatureSection.vue
│   │   ├── ReviewsSection.vue        → fetch: dummyjson.com/users
│   │   └── CustomerPetsSection.vue
│   ├── Login.vue / Signup.vue
│   │   └── AuthComponents/
│   │       ├── AuthPanel.vue
│   │       ├── AuthField.vue
│   │       ├── AuthDivider.vue
│   │       └── AuthGoogleBtn.vue
│   ├── Booking.vue                   → bookings/BookingForm.vue
│   ├── services/
│   │   ├── VetAppointment.vue        → bookings/BookingForm.vue
│   │   ├── EmergencyCare.vue         → bookings/BookingForm.vue
│   │   ├── GroomingBooking.vue       → bookings/BookingForm.vue
│   │   └── TrainingServices.vue      → bookings/BookingForm.vue
│   ├── MyBookings.vue                → useBookingStore() [Pinia]
│   ├── Profile.vue                   → useAuth() + useBookingStore()
│   ├── Marketplace.vue               → fetch: dummyjson.com/products
│   │   ├── ProductDetail.vue         (/marketplace/:id)
│   │   ├── Cart.vue                  (/marketplace/cart)
│   │   └── Orders.vue                (/marketplace/orders)
│   ├── AdminMarketplace.vue          (CRUD over marketplaceStore)
│   ├── About.vue                     → fetch: randomuser.me
│   ├── Contact.vue                   → useContactForm()
│   └── NotFound.vue                  (wildcard route)
└── Footer.vue
```

State & data layer

Module	Type	Responsibility
`stores/bookingStore.ts`	Pinia store	Booking CRUD, grouping by category, localStorage persistence
`stores/marketplaceStore.ts`	Composable module	Marketplace item CRUD, localStorage persistence
`composables/useAuth.ts`	Composable	Login (DummyJSON), logout, current-user lookup
`composables/useToast.ts`	Composable	Global toast queue (success / error / info)
`composables/useContactForm.ts`	Composable	Contact form state, validation, simulated submit
`composables/useScrollReveal.ts`	Composable	Scroll-triggered reveal animations

Note: this is a frontend-only demo. There is no real backend — bookings, cart, orders, and admin-added marketplace items are persisted to browser `localStorage` rather than a database. Authentication uses DummyJSON's public demo auth endpoint.